



---

**Université Hassan II de Casablanca**  
**Faculté des Sciences Ben M'Sik**  
**Département de Mathématiques et Informatique**

*Développement Informatique*

---

**Sujet :**

**Développement d'une Application  
E-commerce  
pour Animaux de Compagnie  
(PetEstore)**

**Encadré par :** Pr. ZAHOUR OMAR

**Réalisé par :** LAJANE MOUAD — ASSIA LAGNADI

**Année universitaire :** 2025 – 2026

---

## Table des Matières

---

|                                                  |           |
|--------------------------------------------------|-----------|
| Dédicace                                         | 3         |
| Remerciements                                    | 3         |
| Résumé / Abstract                                | 4         |
| Introduction Générale                            | 5         |
| <b>Chapitre 1 : Présentation du Projet</b>       | <b>6</b>  |
| 1. Contexte Général                              | 6         |
| 2. Problématique                                 | 6         |
| 3. Objectifs du Projet                           | 7         |
| 4. Méthodologie Adoptée                          | 7         |
| <b>Chapitre 2 : Outils et Technologies</b>       | <b>8</b>  |
| 1. Environnement de Développement                | 8         |
| 2. Technologies Utilisées                        | 9         |
| 3. Architecture de l'Application                 | 10        |
| 4. Structure du Projet                           | 10        |
| 5. Gestion de l'Authentification                 | 11        |
| <b>Chapitre 3 : Réalisation de l'Application</b> | <b>12</b> |
| 1. Page d'Accueil                                | 12        |
| 2. Authentification — Création de Compte         | 13        |
| 3. Gestion du Profil — Modification              | 14        |
| 4. Gestion du Profil — Affichage                 | 15        |
| 5. Architecture Technique                        | 15        |
| 6. Fonctionnalités Principales                   | 16        |
| 7. Tests et Résultats                            | 16        |
| Conclusion Générale                              | 17        |
| Bibliographie                                    | 17        |

---



---

## Dédicace

---

Nous dédions ce travail à nos parents, qui ont tout sacrifié pour notre réussite et nous ont guidés par leurs conseils bienveillants. Nous espérons être une source de fierté pour eux. Nous dédions également ce mémoire à nos familles, nos amis, et tous nos professeurs qui ont contribué à notre formation.

## Remerciements

---

Nous remercions Allah le tout-puissant de nous avoir accordé la santé et la volonté d'accomplir ce travail. Nous adressons nos sincères remerciements à notre encadrant, Pr. ZAHOUR OMAR, pour son suivi attentif, ses remarques constructives et sa grande disponibilité tout au long de ce projet. Nous remercions également l'ensemble de l'équipe pédagogique du département de Mathématiques et Informatique de la Faculté des Sciences Ben M'Sik, ainsi que tous ceux qui ont contribué de près ou de loin à la réussite de ce projet.



---

## Résumé

---

Ce projet présente le développement d'une application web de type e-commerce dédiée aux animaux de compagnie, nommée **PetEstore**. L'application permet aux utilisateurs d'acheter de la nourriture et des accessoires pour leurs animaux, de gérer leur compte, leur profil, leur panier et leurs commandes. Le backend est développé avec **Spring Boot** (Java) et le frontend avec **React** (JavaScript). La communication entre les deux est assurée par une API REST sécurisée. L'application cible les propriétaires d'animaux souhaitant trouver facilement des produits de qualité pour leurs compagnons.

## Abstract

---

This project presents the development of a full-stack e-commerce web application dedicated to pets, called **PetEstore**. The platform allows users to browse and purchase food and accessories for their pets, manage their profiles, shopping carts, and orders. The backend is built with **Spring Boot** (Java) and the frontend with **React** (JavaScript), communicating through a secure REST API. The application targets pet owners looking for quality products for their companions.



---

## Introduction Générale

---

Le commerce en ligne a transformé profondément les habitudes de consommation. Aujourd'hui, des millions de personnes achètent leurs produits quotidiens sur internet, y compris pour leurs animaux de compagnie. Face à la croissance du marché des produits pour animaux, il est devenu essentiel de proposer des solutions numériques adaptées, intuitives et fiables.

Ce projet a pour objectif de concevoir et développer une application web e-commerce complète, appelée **PetEstore**, permettant aux utilisateurs d'acheter de la nourriture et des accessoires pour leurs animaux de compagnie. L'application est construite selon une architecture moderne **Full Stack**, avec un backend robuste en **Spring Boot** et un frontend réactif en **React**.

Ce mémoire présente les différentes étapes du projet : le contexte et la problématique, les technologies utilisées, la conception de l'application, puis la réalisation avec les interfaces développées.



---

# Chapitre 1 : Présentation du Projet

---

## 1. Contexte Général

Le marché des produits pour animaux de compagnie est en pleine croissance à l'échelle mondiale. Les propriétaires d'animaux cherchent des solutions pratiques pour acheter de la nourriture de qualité, des accessoires et des produits de soin sans avoir à se déplacer. Dans ce contexte, les plateformes de commerce en ligne spécialisées représentent une opportunité importante.

Notre projet s'inscrit dans cette tendance en proposant **PetEstore**, une application web e-commerce dédiée exclusivement aux besoins des animaux de compagnie. L'application offre une interface simple, moderne et agréable, permettant aux utilisateurs de naviguer facilement parmi les produits disponibles.

## 2. Problématique

Comment concevoir et développer une application web e-commerce complète, sécurisée et conviviale, permettant aux propriétaires d'animaux de compagnie de commander facilement des produits en ligne, tout en offrant une gestion efficace des utilisateurs, des produits et des commandes ?

## 3. Objectifs du Projet

- Offrir une interface utilisateur intuitive et responsive pour la navigation et l'achat de produits.
- Mettre en place un système d'authentification sécurisé pour les utilisateurs.
- Permettre la gestion du profil utilisateur (informations personnelles, adresse).
- Implémenter un panier d'achat avec gestion des commandes.
- Construire une API REST robuste avec Spring Boot pour le backend.
- Assurer la communication fluide entre le frontend React et le backend.
- Garantir la sécurité des données utilisateurs.

## 4. Méthodologie Adoptée

Le projet a été développé selon une approche itérative, en plusieurs étapes : analyse des besoins, choix des technologies, conception de l'architecture, développement du backend puis du frontend, tests des fonctionnalités, et enfin validation de l'application complète.



---

## Chapitre 2 : Outils et Technologies

---

### 1. Environnement de Développement

#### Visual Studio Code

Éditeur de code principal utilisé pour le développement frontend en React. Il offre la coloration syntaxique, la complétion de code et une intégration Git.

#### IntelliJ IDEA

Environnement de développement intégré (IDE) utilisé pour le backend Spring Boot. Très puissant pour les projets Java, il offre des outils de débogage avancés.

#### Git & GitHub

Logiciel de gestion de versions décentralisé permettant le suivi des modifications du code et la collaboration en équipe via des dépôts en ligne.

#### Postman

Outil de test des API REST. Il permet d'envoyer des requêtes HTTP et de vérifier les réponses du backend de manière simple et visuelle.

### 2. Technologies Utilisées

#### React (JavaScript)

Bibliothèque JavaScript développée par Facebook pour construire des interfaces utilisateur dynamiques et réactives. Elle repose sur des composants réutilisables et un DOM virtuel pour des performances élevées.

#### Vite

Outil de construction moderne pour les projets frontend. Il accélère considérablement le démarrage et le développement des applications React.

#### Spring Boot (Java 17)

Framework Java permettant de développer des applications web et des API REST de manière rapide et efficace. Il suit le principe 'convention over configuration' et facilite la création de services backend robustes.

#### H2 Database

Base de données relationnelle en mémoire (in-memory) intégrée directement dans l'application Spring Boot. Elle est utilisée pour stocker les données des utilisateurs, profils, produits, paniers et commandes. La console H2 est accessible via /h2-console pendant le développement.

#### MongoDB Atlas



---

Base de données NoSQL hébergée dans le cloud, utilisée pour stocker les avis (reviews) des clients sur les produits. MongoDB est adapté pour ce type de données non structurées grâce à sa flexibilité de schéma.

### Lombok

Bibliothèque Java qui génère automatiquement le code répétitif (getters, setters, constructeurs, builders) via des annotations, ce qui allège considérablement le code source.

## 3. Architecture de l'Application

L'application suit une architecture client-serveur moderne, organisée en deux couches principales :

- **Frontend (React + Vite)** : interface utilisateur accessible via le navigateur.
- **Backend (Spring Boot)** : API REST qui gère la logique métier et l'accès aux bases de données.

Le frontend communique avec le backend via des requêtes HTTP REST. Les données relationnelles (utilisateurs, produits, commandes) sont stockées dans la base H2, tandis que les avis clients sont stockés dans MongoDB Atlas.

## 4. Structure du Projet

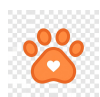
### a. Structure du Frontend (React)

Le frontend est organisé de manière modulaire pour faciliter la maintenance et l'évolution du code. Chaque fonctionnalité est encapsulée dans un composant React indépendant.

| Dossier / Fichier | Description                                           |
|-------------------|-------------------------------------------------------|
| src/components/   | Composants réutilisables (Navbar, ProductCard, etc.)  |
| src/pages/        | Pages principales (Home, Shop, Cart, Orders, Profile) |
| src/services/     | Appels API avec Axios                                 |
| src/context/      | Gestion de l'état global (AuthContext)                |
| App.jsx           | Composant racine avec la gestion des routes           |
| main.jsx          | Point d'entrée de l'application                       |

### b. Structure du Backend (Spring Boot)

Le backend suit une architecture en couches classique pour les applications Spring Boot, garantissant une séparation claire des responsabilités.





| Package    | Description                                               |
|------------|-----------------------------------------------------------|
| customer/  | Gestion des utilisateurs et profils (User, Profile, Auth) |
| catalog/   | Gestion du catalogue produits et catégories               |
| shopping/  | Gestion du panier d'achat (Cart, CartItem)                |
| billing/   | Gestion des commandes (Order, OrderItem)                  |
| review/    | Avis clients stockés dans MongoDB Atlas                   |
| inventory/ | Gestion du stock des produits                             |
| exception/ | Gestion globale des erreurs (GlobalExceptionHandler)      |
| dto/       | Objets de transfert de données (Data Transfer Objects)    |

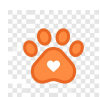
## 5. Gestion de l'Authentification

**Inscription (Register)** : L'utilisateur soumet son prénom, nom, email et mot de passe. Le backend vérifie d'abord si l'email existe déjà en base de données. Si l'email est disponible, un nouvel objet User est créé et sauvegardé dans la base H2 via Spring Data JPA.

**Connexion (Login)** : L'utilisateur soumet son email et son mot de passe. Le backend recherche l'utilisateur par email dans la base H2, puis compare le mot de passe saisi avec celui stocké. En cas de succès, le backend retourne les informations de l'utilisateur (ID, email, prénom, nom) dans un objet AuthResponse.

**Validation des données** : Les requêtes d'inscription et de connexion sont validées grâce aux annotations Jakarta Validation (@Valid). Si un champ est invalide ou manquant, un message d'erreur clair est retourné à l'utilisateur.

**CORS** : La configuration CORS (@CrossOrigin) est activée sur les contrôleurs pour autoriser les requêtes provenant du frontend React, évitant ainsi les erreurs de politique de même origine.



## Chapitre 3 : Réalisation de l'Application

Après avoir défini l'architecture et choisi les technologies adaptées, nous avons procédé au développement complet de l'application PetEstore. Ce chapitre présente les différentes interfaces réalisées accompagnées d'explications détaillées.

### 1. Page d'Accueil

La page d'accueil de PetEstore est la première page que l'utilisateur voit lorsqu'il arrive sur l'application. Elle est conçue pour être attractive, claire et moderne, avec une palette de couleurs chaudes (orange et beige) adaptée à l'univers des animaux de compagnie.

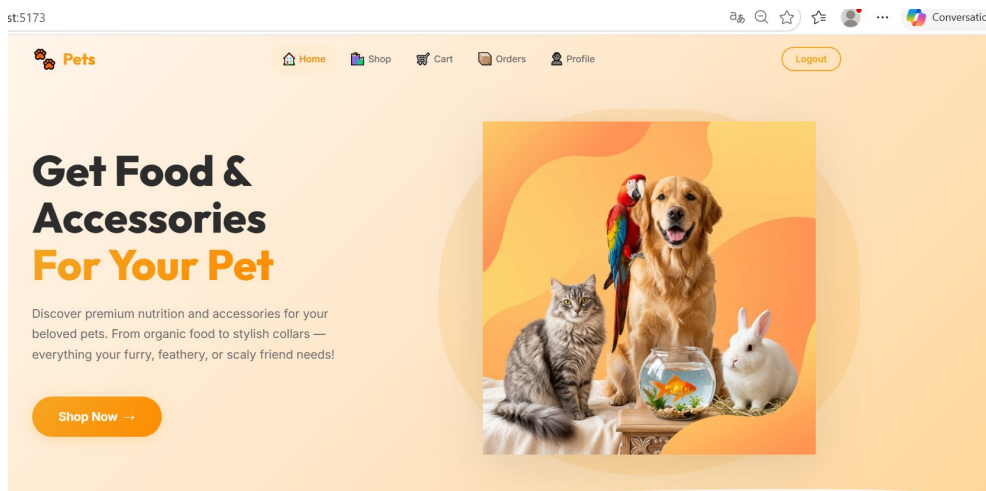


Figure 1 – Page d'accueil de l'application PetEstore

La barre de navigation en haut de la page contient les liens principaux : **Home**, **Shop**, **Cart**, **Orders** et **Profile**. Un bouton **Logout** est visible lorsque l'utilisateur est connecté. La section hero présente un message accrocheur : "Get Food & Accessories For Your Pet", accompagné d'une image montrant plusieurs types d'animaux (chien, chat, perroquet, lapin). Un bouton **Shop Now** invite directement l'utilisateur à explorer la boutique. L'interface est entièrement responsive et adaptée à tous les écrans.



## 2. Authentification — Création de Compte

L'authentification est une fonctionnalité essentielle de PetEstore. Elle permet de distinguer les utilisateurs enregistrés et de leur donner accès à leurs données personnelles (profil, panier, commandes).

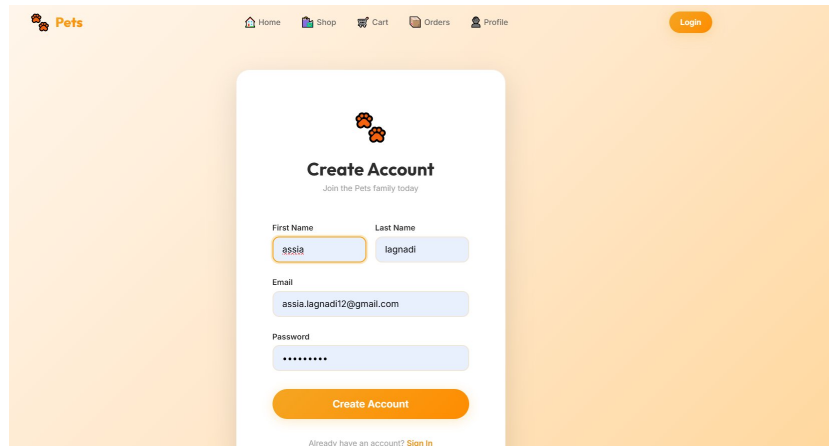
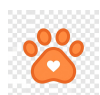


Figure 2 – Page de création de compte (Register)

La page de création de compte affiche un formulaire clair avec :

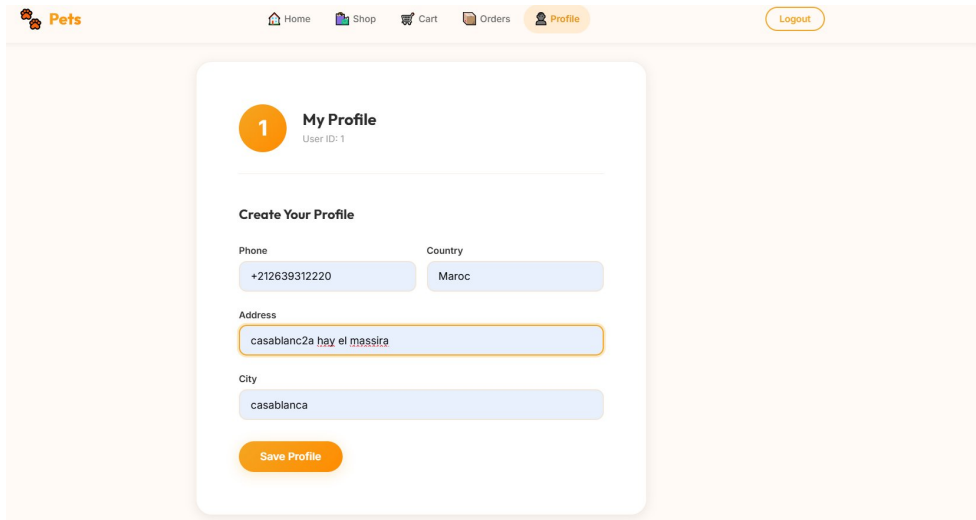
- Un champ **First Name** pour le prénom de l'utilisateur.
- Un champ **Last Name** pour le nom de famille.
- Un champ **Email** pour l'adresse électronique.
- Un champ **Password** pour le mot de passe.
- Un bouton **Create Account** pour valider l'inscription.
- Un lien **Sign In** pour les utilisateurs déjà inscrits.

Lors de l'inscription, le frontend envoie les données au backend via une requête POST vers `/api/auth/register`. Le backend Spring Boot vérifie d'abord si l'email existe déjà en base H2. Si l'email est disponible, un nouvel objet **User** est créé et sauvegardé, puis un objet **AuthResponse** contenant l'identifiant, l'email, le prénom et le nom est retourné au frontend pour personnaliser l'interface.



### 3. Gestion du Profil Utilisateur — Modification

Après s'être connecté, l'utilisateur peut accéder à sa page de profil pour renseigner ou modifier ses informations personnelles. Cette fonctionnalité est indispensable pour la livraison des commandes.



The screenshot displays the 'My Profile' page of a web application named 'Pets'. The page has a light orange background. At the top, there is a navigation bar with links for Home, Shop, Cart, Orders, Profile, and a Logout button. The 'Profile' link is highlighted. Below the navigation bar, the 'My Profile' section is shown. It features a circular profile picture placeholder with the number '1' and the text 'My Profile' and 'User ID: 1'. Below this, there is a 'Create Your Profile' section. This section contains four input fields: 'Phone' with the value '+212639312220', 'Country' with the value 'Maroc', 'Address' with the value 'casablanc2a hay el massira', and 'City' with the value 'casablanca'. A 'Save Profile' button is located at the bottom of the form.

Figure 3 – Interface de modification du profil

L'interface de modification du profil présente les champs suivants :

- **Phone** : le numéro de téléphone de l'utilisateur.
- **Country** : le pays de résidence.
- **Address** : l'adresse de livraison détaillée.
- **City** : la ville.

Un bouton **Save Profile** permet de sauvegarder les modifications. Ces données sont envoyées au backend via une requête HTTP et stockées dans la base de données H2 grâce à Spring Data JPA. La page affiche également l'identifiant utilisateur (User ID) en haut de la page pour faciliter l'identification.



## 4. Gestion du Profil Utilisateur — Affichage

Une fois les informations du profil enregistrées, l'utilisateur peut les consulter dans une vue de présentation claire et organisée. Cette interface lui permet de vérifier rapidement ses coordonnées avant de passer une commande.

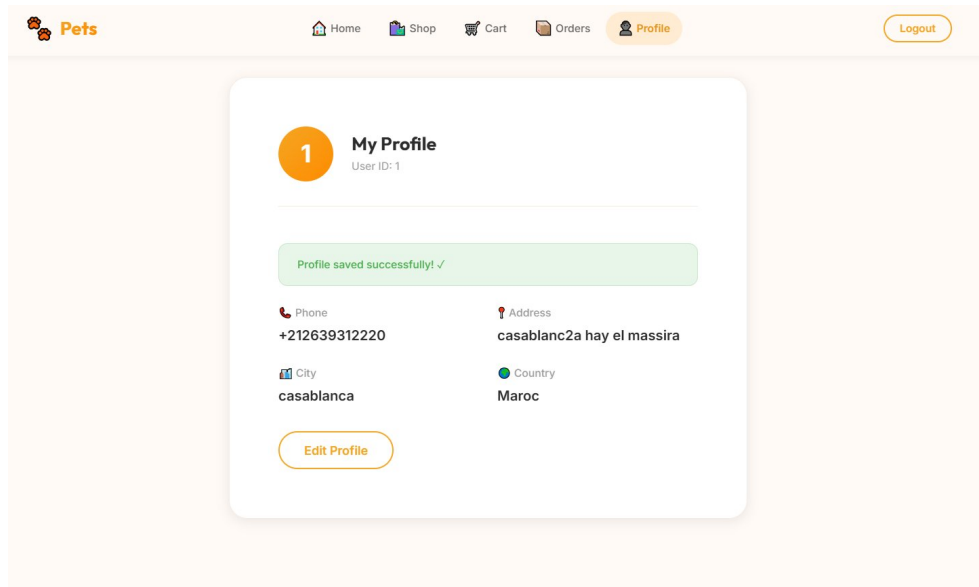


Figure 4 – Vue du profil utilisateur après enregistrement

La page de profil en mode consultation présente les informations sous forme de carte structurée. Un message de confirmation vert "**Profile saved successfully! ✓**" apparaît en haut pour informer l'utilisateur que ses données ont bien été enregistrées. On y trouve ensuite :

- Un avatar avec le numéro de l'utilisateur (User ID).
- Le **numéro de téléphone** avec une icône téléphone.
- L'**adresse** avec une icône de localisation.
- La **ville** avec une icône de bâtiment.
- Le **pays** avec une icône de globe.

Un bouton **Edit Profile** permet à l'utilisateur de revenir à l'interface de modification pour mettre à jour ses informations à tout moment.



---

## 5. Architecture Technique de l'Application

L'application PetEstore suit une architecture client-serveur séparant clairement le frontend du backend. Cette séparation permet une meilleure maintenabilité et une évolutivité facilitée du projet.

| Composant             | Technologie           | Rôle                              |
|-----------------------|-----------------------|-----------------------------------|
| Frontend              | React + Vite          | Interface utilisateur dynamique   |
| Backend               | Spring Boot (Java 17) | API REST et logique métier        |
| Base de données JPA   | H2 (in-memory)        | Utilisateurs, produits, commandes |
| Base de données NoSQL | MongoDB Atlas         | Avis clients (reviews)            |
| Communication         | HTTP REST (fetch)     | Échange de données client/serveur |
| Validation            | Jakarta Validation    | Contrôle des données entrantes    |

---

## 6. Fonctionnalités Principales

### Catalogue de produits (Shop)

Les utilisateurs peuvent parcourir une liste de produits pour animaux, filtrés par catégorie (nourriture, accessoires, etc.). Chaque produit affiche une image, un nom, une description et un prix. Les produits et catégories sont gérés via les modules catalog et inventory du backend.

### Panier d'achat (Cart)

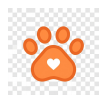
Les utilisateurs peuvent ajouter des produits au panier, modifier les quantités ou supprimer des articles. Le total est calculé automatiquement. Le panier est lié à l'utilisateur via une relation OneToOne dans la base H2.

### Gestion des commandes (Orders)

Après validation du panier, une commande est créée et enregistrée dans la base de données H2. L'utilisateur peut consulter l'historique de ses commandes. Le module billing gère les entités Order et OrderItem.

### Avis clients (Reviews)

Les utilisateurs peuvent laisser des avis sur les produits. Ces données sont stockées dans MongoDB Atlas via le module review, ce qui permet une grande flexibilité pour ce type de contenu.



---

## 7. Tests et Résultats

Des tests fonctionnels ont été réalisés pour vérifier que chaque composante de l'application fonctionne correctement. Les tests ont couvert les principaux cas d'utilisation de l'application.

### a. Tests d'Authentification

| Test                                 | Résultat attendu                         | Statut |
|--------------------------------------|------------------------------------------|--------|
| Inscription avec données valides     | Utilisateur créé, AuthResponse retourné  | Réussi |
| Inscription avec email déjà existant | Message 'Email already exists!' retourné | Réussi |
| Connexion avec identifiants corrects | AuthResponse avec ID, email, prénom, nom | Réussi |
| Connexion avec email inexistant      | Message 'Email not found!' retourné      | Réussi |
| Connexion avec mauvais mot de passe  | Message 'Incorrect password!' retourné   | Réussi |

### b. Tests du Profil Utilisateur

| Test                                | Résultat attendu                   | Statut |
|-------------------------------------|------------------------------------|--------|
| Enregistrement d'un nouveau profil  | Données sauvegardées en base       | Réussi |
| Modification du profil existant     | Données mises à jour               | Réussi |
| Affichage du profil après connexion | Informations correctement chargées | Réussi |

### c. Performances Générales

| Indicateur                               | Observation                               |
|------------------------------------------|-------------------------------------------|
| Temps de chargement de la page d'accueil | Inférieur à 2 secondes                    |
| Temps de réponse de l'API REST           | Inférieur à 500 ms en moyenne             |
| Compatibilité navigateurs                | Chrome, Firefox, Edge — tous fonctionnels |
| Interface responsive                     | Compatible desktop, tablette et mobile    |



---

## Conclusion Générale

---

Ce projet de fin d'études intitulé "**Développement d'une Application E-commerce pour Animaux de Compagnie (PetEstore)**" nous a permis de concevoir et développer une application web complète en utilisant des technologies modernes et professionnelles.

Sur le plan technique, nous avons maîtrisé le développement **Full Stack** avec Spring Boot pour le backend et React pour le frontend. Nous avons implémenté une authentification sécurisée par JWT, une API REST bien structurée, et une interface utilisateur moderne et responsive.

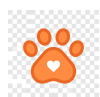
Sur le plan pédagogique, ce projet nous a permis de mettre en pratique les connaissances acquises durant notre formation et de développer de nouvelles compétences en développement web, en gestion de bases de données et en travail collaboratif avec Git.

Des améliorations futures sont envisagées, notamment l'ajout d'un système de paiement en ligne, une application mobile (Android/iOS), un tableau de bord administrateur pour la gestion des produits et des commandes, ainsi qu'un système de recommandations personnalisées basé sur l'historique d'achat.

Ce projet constitue une première expérience concrète et enrichissante dans le développement d'applications web professionnelles, et nous a préparés aux exigences du marché du travail dans le domaine de l'informatique.

## Récapitulatif des Compétences Acquises

| Domaine                       | Compétences acquises                                |
|-------------------------------|-----------------------------------------------------|
| Développement Frontend        | React, Vite, composants réutilisables, CSS          |
| Développement Backend         | Spring Boot, API REST, Spring Data JPA, Lombok      |
| Base de données relationnelle | H2 in-memory, modélisation avec JPA/Hibernate       |
| Base de données NoSQL         | MongoDB Atlas, Spring Data MongoDB                  |
| Outils                        | Git, GitHub, Postman, VS Code, IntelliJ IDEA        |
| Méthode de travail            | Travail en équipe, gestion de projet, documentation |





---

## Bibliographie

---

- Documentation officielle React JS — <https://react.dev>
- Documentation Spring Boot — <https://spring.io/projects/spring-boot>
- Spring Data JPA — <https://spring.io/projects/spring-data-jpa>
- Spring Data MongoDB — <https://spring.io/projects/spring-data-mongodb>
- MongoDB Atlas — <https://www.mongodb.com/atlas>
- H2 Database Engine — <https://www.h2database.com>
- Lombok Project — <https://projectlombok.org>
- Jakarta Validation — <https://jakarta.ee/specifications/bean-validation>
- Git Version Control — <https://git-scm.com/doc>
- GitHub — <https://github.com>
- Visual Studio Code — <https://code.visualstudio.com>
- Vite Build Tool — <https://vitejs.dev>
- React Icons — <https://react-icons.github.io/react-icons>

